中国大学生计算机设计大赛

软件开发类作品文档简要要求

作品编号：2026017676

作品名称：轻巡智维：道路病害轻量化智能巡检系统

版本编号：v1.0

填写日期：2026 年 4 月 18 日

目 录

第一章： 需求分析 1

1.1 开发背景 1

1.2 竞品分析 1

1.3 用户需求 2

第二章： 概要设计 2

2.1 系统总体架构 2

2.2 功能模块划分 3

2.3 模块调用关系 3

2.4 模块间接口 3

2.5 人机界面与典型交互流程 4

第三章： 详细设计 4

3.1 界面设计 4

3.2 数据库设计 5

3.3 关键技术创新 7

第四章： 测试报告 9

4.1 模型检测性能测试 9

4.2 接口性能测试 11

4.3 关键功能测试 12

4.4 技术指标汇总 12

第五章： 安装及使用 12

5.1 环境要求 12

5.2 安装过程 13

5.3 典型使用流程 14

第六章： 项目总结 14

6.1 开发历程与团队协作 14

6.2 攻克的主要难点 14

6.3 后续演进方向 15

参考文献 15

第一章：需求分析

1.1 开发背景

中国公路网已全面进入”建养并重”的存量维护时代。据交通运输部《2024 年交通运输行业发展统计公报》[1]，2024 年末全国公路总里程达 549 万公里，高速公路 19.07 万公里，均居世界第一；2025 年公路养护市场规模达 1.25 万亿元，交通基础设施巡检平台市场规模预计突破 800 亿元，年复合增长率达 18%–20%。

政策层面持续加码。《“十四五”公路养护管理发展纲要》[2] 明确提出路面自动化采集覆盖率达 100% 的目标；2025 年 9 月，交通运输部等七部门联合印发《关于”人工智能＋交通运输”的实施意见》[3]，在”智能建养”部分明确提出”推广应用轻量化智能路面检测车”，”推进人工智能技术在交通基础设施日常巡查、检测监测、病害诊断、技术状况评定、养护决策等应用”，为路面缺陷 AI 智能检测的落地提供了明确的顶层支撑。

然而现实存在显著落差。《公路技术状况评定标准》（JTG 5210-2018）[4] 规定高速公路每年检测两次、普通国省干线一至两次，但专业检测车单台采购价 200–350 万元，检测外包费 1000–1200 元/公里·车道，全国公路养护资金缺口约 50%，大量路段实际检测频次严重不足，病害从萌芽到恶化的演变过程长期处于监测盲区。

更深层的矛盾在于**数据来源的结构性匮乏**。现有方案均以专用检测设备为唯一采集入口，而每日在公路上高频往返的公交车、环卫车、出租车、邮政车已内置行车记录仪，无人机、道路监控摄像头亦广泛部署，市民与巡检人员随时携带智能手机——这些**存量数据资产**因缺乏统一接入与处理能力而长期沉睡。多源设备在坐标系、时间戳、图像质量上存在异构性，**多源数据的融合与空间对齐**是将其转化为可用巡检数据的核心技术挑战。

近年来，随着计算机视觉技术的飞速发展，深度学习在路面病害检测领域已取得显著进展，但在复杂光照、遮挡等极端环境下的鲁棒性仍是当前学术界的研究热点 [5][6][7][8]。传统的检测方法往往依赖于单一传感器，而本项目则尝试在轻量化架构下实现更高精度的突破。

1.2 竞品分析

市场现有方案均需专用硬件，核心差距如表 1所示。

表 1: 主要竞品对比

产品	硬件门槛	单套成本	病害追踪	工单闭环	众包采集
中数途·高精度三维专业检测车	专用改装车 + 持证检测员	200–350 万/台	无跨次关联	需外部对接	×
千寻位置·千寻驰观轻量化巡检	专用 AI 相机 + 边缘计算盒	≈10 万/套	无跨次聚合	需外部对接	×
山西交控·AI+ 北斗智能巡检装备	北斗模组 + 专用相机	≈10 万/套	无跨次聚合	需外部对接	×
湖北交投·空地一体化 AI 巡检系统	集成巡检套件 + 无人机场	百万级	无跨次聚合	需外部对接	×
云圣智能·路空一体立体交通巡检	无人机场网络	30–50 万/套机场	无跨次聚合	需外部对接	×
LightScan（本作品）	**无，复用手机/行车记录仪**	**零硬件成本**	**GPS+ 视觉 ReID 聚类**	**内置全生命周期**	✓

现有方案聚焦”精确检测”，共同缺陷是硬件门槛高、数据跨次孤立、检测结果与工单管理割裂。LightScan 定位于**无专用硬件条件下的高频次、可追踪、有闭环常态化巡检**，填补县乡级管养单位的技术空白。系统支持手机拍照、行车记录仪视频（公交车、无人机、私家车等）、巡检员上传与市民众包等多元采集入口，通过时空约束与视觉 ReID 融合算法实现跨源数据的统一对齐与去重聚类，将存量公共车辆与市民资源转化为常态化巡检网络。

1.3 用户需求

根据上述背景，系统面向三类角色的核心诉求如下：

- **市民**：发现路面问题后需要一个**零门槛的上报渠道**，无需注册、无需专业判断，10 秒内完成拍照提交即可。
- **基层巡检员**：日常巡检中**不应额外采购设备**，手机与行车记录仪应能直接作为采集工具；病害识别与定位应自动化，减少人工判读负担；工单分派、处理与核销需有清晰的流转记录。
- **养护管理部门**：需掌握辖区病害的**空间全貌与演变趋势**，而非仅有离散的单次检测快照；需要从" 发现病害" 到" 修复核验" 的**闭环管理**，确保每条工单有人负责、有结果可查；高频次的多源数据汇聚应能支撑**预防性养护决策**，在病害恶化前介入。

由此归纳四项核心需求：

R1. 多源异构数据统一接入：手机、行车记录仪、无人机及市民众包数据的空间对齐与标准化处理；

R2. 轻量化实时识别：无专用硬件依赖的检测引擎，降低技术与经济门槛；

R3. 时空演变追踪：跨批次、跨来源的病害点自动聚合，支持病害生命周期全过程监测；

R4. 全生命周期闭环：覆盖病害发现、工单分发到修复验收的数字化管理流程。

第二章：概要设计

2.1 系统总体架构

针对第一章归纳的四项核心需求（R1–R4），系统采用前后端分离的四层架构，各端通过 RESTful API 与云端服务层统一通信，如图 1 所示。

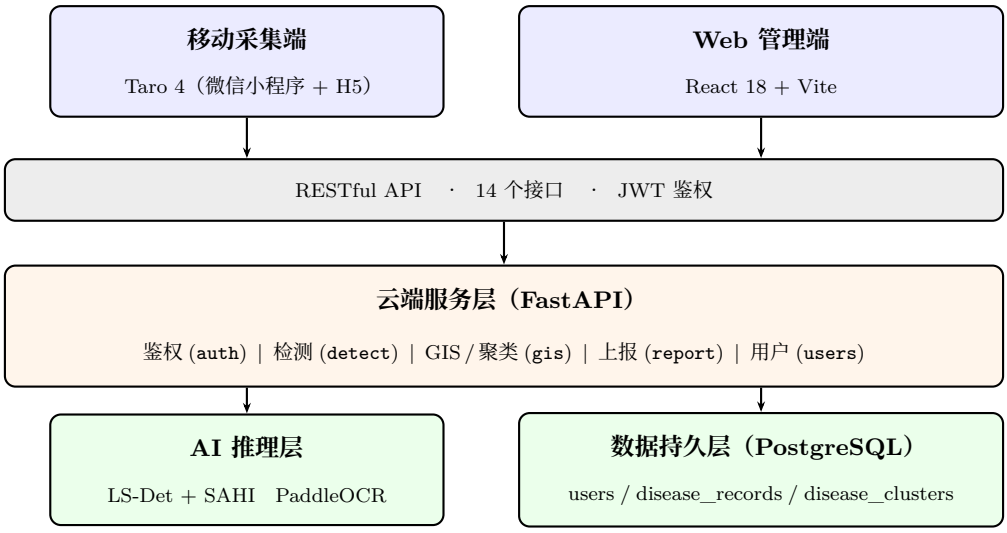

图 1: LightScan 系统总体架构

2.2 功能模块划分

表 2: 功能模块与需求映射

层级	模块	主要职责	对应需求
移动采集端	市民上报	选择病害类型、拍照上传、GPS 自动采集，无需注册	R1
	巡检员工作台	查看工单列表、按状态筛选、处理并上传修复照片	R4
Web 管理端	图片检测	批量上传图片（≤20 张），展示标注结果，一键入库	R2
	视频检测	上传行车记录仪视频，OCR/估算模式按里程抽帧检测	R1、R2
	GIS 地图	散点/热力图/时间轴三层渲染，病害点工单原地流转	R3、R4
	数据大屏	3D 态势感知，展示病害分布、来源与趋势统计	R3
云端服务层	鉴权模块	JWT 签发与校验、bcrypt 密码哈希、角色管理	—
	AI 报告引擎	聚合病害统计摘要，调用 DeepSeek [15] 生成巡检周报与单点养护建议	R3、R4
	检测与聚类	LS-Det 推理、EXIF/OCR 定位、ReID 特征提取、时空聚类	R1、R2、R3
	上报与工单	众包记录入库、工单状态流转、软删除管理	R4
AI 推理层	LS-Det	单例懒加载，GPU/CPU 自动切换，4 类病害实时检测	R2
	PaddleOCR	识别视频帧速度叠字，供视频服务按里程等距抽帧	R1

2.3 模块调用关系

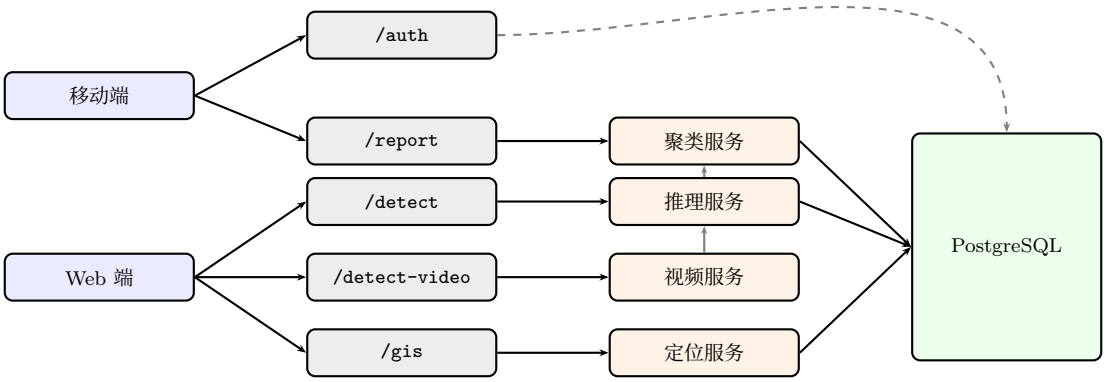

图 2: 模块调用关系图（虚线表示直连数据库，灰箭头表示服务内部调用）

2.4 模块间接口

表 3: 主要模块间接口

接口端点	调用方	输入	输出
`POST /auth/login`	Web / 移动端	用户名 + 密码	JWT Token
`POST /detect`	Web 端	图片文件 + 置信度阈值	标注图 + 检测框列表
`POST /detect-video`	Web 端	MP4 文件 + 模式参数	逐帧检测结果 + 坐标列表
`POST /report`	移动端	图片 + GPS 坐标	记录 ID + 聚类 ID
`GET /gis/records`	Web 端	JWT（鉴权）	全量病害点 GeoJSON
`PATCH /gis/records/{id}/status`	Web / 移动端	新状态 + 操作人	更新后记录
`GET /gis/clusters/{id}/timeline`	Web 端	聚类 ID	该病害点历次检测时间轴
`POST /ai/report`	Web 端	时间窗口 + 区域标签	AI 生成的中文巡检周报

2.5 人机界面与典型交互流程

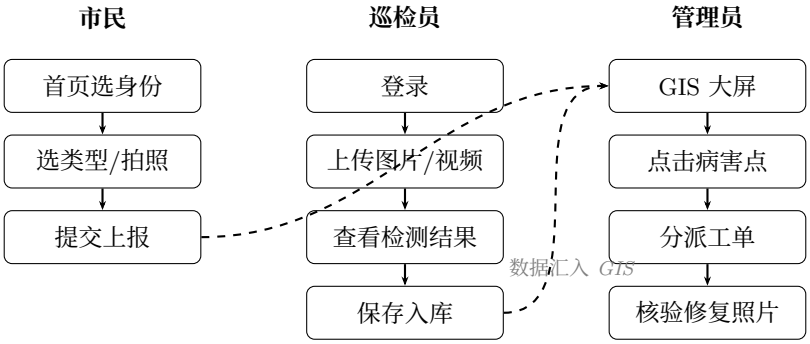

图 3: 三类用户典型交互流程

第三章：详细设计

3.1 界面设计

3.1.1 视觉规范与原子化设计

系统以**科技蓝（#3E6AE1）**为品牌色体现技术感，底色采用**灰白（#F4F4F4）**，卡片与分割线选用 **#EEEEEE**，三色构成完整色阶，有意克制色彩数量以降低外业长时作业的视觉疲劳。排版采用“字重驱动”策略：关键数字与标题以 700–800 字重加粗，辅助信息采用 400 字重常规体，信息层级通过字重而非颜色进行区分。Web 与移动端共用一套设计令牌，由 Taro + React 原子组件保证跨端视觉的高度一致性 [18]。

3.1.2 Web 管理端与移动端设计决策

- **Web 管理端**：围绕“检测 → 入库 → 管理”三段式链路设计。检测面板支持批量拖拽上传，以“标注预览 + 属性卡片”并排呈现结果；GIS 大屏以散点、热力图、时间轴三层叠加，严重度映射为“黄–橙–红–品红”色阶，实现风险分布的一瞥即知；工单状态流转直接嵌入地图 InfoWindow 内，无需页面跳转。

3.1.3 移动端：差异化交互与角色驱动

移动端根据用户角色采取了截然相反的设计决策，体现了“角色感知”的核心思路：

- **市民端 (众包模式)**：以 **“零门槛、无感知”**为设计目标。采用沉浸式表单，病害类型通过大尺寸图标宫格进行一触式选择，支持拍照直传。系统全程无需用户注册登录，通过隐形定位技术自动对齐坐标，将上报操作耗时压缩至 10 秒以内。

- **巡检员端 (专业模式)**：以 **“高信息密度”**为设计目标。首页顶部设置实时统计横条，通过状态徽章和即时响应的 Tab 筛选器，确保巡检员能快速定位待办任务。卡片式列表在有限空间内聚合了位置、类型、时限三维关键信息。

3.1.4 典型业务流程设计

系统交互以“感知 → 决策 → 闭环”为核心链路。下图展示了从影像输入到工单闭环的完整使用流程。

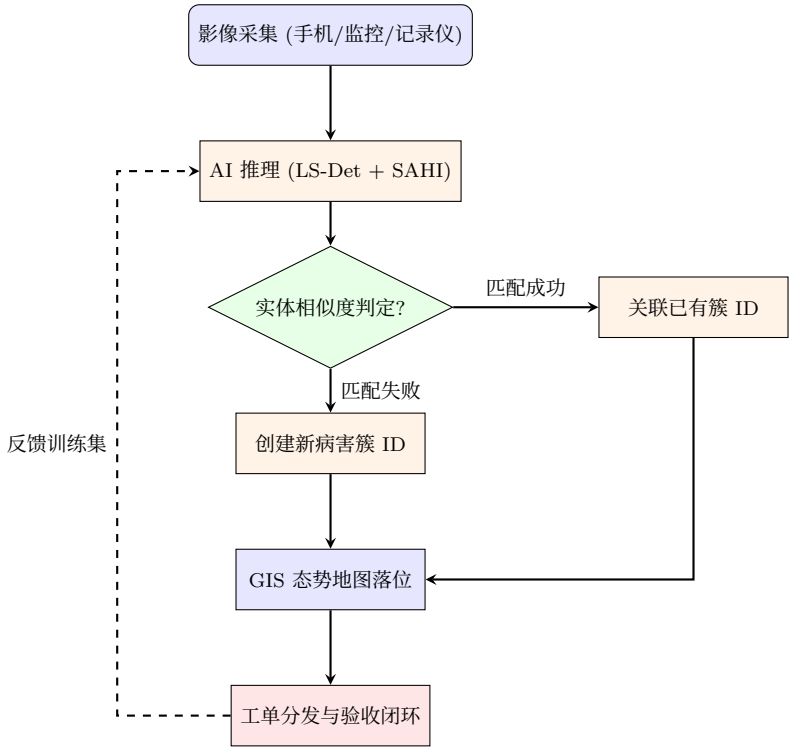

图 4: LightScan 业务流转与算法协同全流程图

3.2 数据库设计

3.2.1 表结构详细设计

系统采用 PostgreSQL 进行数据持久化，共设计三张核心表。针对病害记录的高频空间查询特性，对地理坐标及 `cluster_id`、`label_cn` 字段建立索引。

表 4: 核心表 1：users（用户信息表）

字段名	数据类型	约束	说明
id	INTEGER	PRIMARY KEY	用户唯一标识
username	VARCHAR	UNIQUE, NOT NULL	登录用户名
hashed_password	VARCHAR	NOT NULL	bcrypt 哈希密码
role	VARCHAR	NOT NULL	权限角色（admin / worker）
nickname	VARCHAR	—	显示名称
unit	VARCHAR	—	所属单位
source_type	VARCHAR	—	设备类型（手持 / 行车记录仪 / 无人机）
device_id	VARCHAR	—	设备编号
is_active	BOOLEAN	DEFAULT TRUE	账号启用状态
created_at	DATETIME	DEFAULT NOW()	注册时间

表 5: 核心表 2：disease_clusters（病害聚类主实体表）

字段名	数据类型	约束	说明
cluster_id	UUID	PRIMARY KEY	病害点唯一标识（系统生成）
worker_id	INTEGER	FK → users.id	负责巡检员
label_cn	VARCHAR	NOT NULL	主要病害类型
canonical_lat / lng	FLOAT	NOT NULL	病害点质心坐标
status	VARCHAR	DEFAULT 'pending'	工单状态（待处理 / 处理中 / 已修复）
detection_count	INTEGER	DEFAULT 1	累计检测次数
first_detected_at	DATETIME	NOT NULL	首次发现时间
last_detected_at	DATETIME	NOT NULL	最近一次检测时间
repaired_at	DATETIME	—	修复完成时间
repaired_image_b64	TEXT	—	修复后照片（Base64）

表 6: 核心表 3：disease_records（病害检测记录表）

字段名	数据类型	约束	说明
id	SERIAL	PRIMARY KEY	记录自增主键
cluster_id	UUID	FK → disease_clusters, INDEX	所属病害聚类
creator_id	INTEGER	FK → users.id	上报用户
filename	VARCHAR	NOT NULL	来源图片文件名
timestamp	DATETIME	NOT NULL	采集上报时间
lat / lng	FLOAT	INDEX, NOT NULL	WGS-84 地理空间坐标
label_cn	VARCHAR	NOT NULL	病害中文类别
confidence	FLOAT	—	LS-Det 模型置信度分数
bbox	JSONB	—	检测框像素坐标 [x1,y1,x2,y2]
feature_vector	JSONB	—	32 维 HSV 特征指纹（用于 ReID 匹配）
source_type	VARCHAR	—	数据来源（移动上报 / 视频帧 / 图片检测）
deleted_at	DATETIME	DEFAULT NULL	软删除时间戳，NULL 表示记录有效

3.2.2 反范式设计说明

本系统在 `disease_records` 表中冗余存储了本应属于 `disease_clusters` 表的 `label_cn` 字段，并直接存储了高维特征指纹 `feature_vector`。该违背第三范式的设计基于以下理由：

1. **查询性能优化**：在 GIS 态势感知大屏渲染万级病害散点时，若通过多表关联（JOIN）获取病害类型信息，会产生巨大的计算开销与 I/O 延迟。通过冗余存储，系统可实现单表索引覆盖查询，确保前端地图加载与缩放的极致流畅感。

2. **聚类算法响应速度**：基于 ReID 的视觉聚类算法需要极高频地比对历史特征向量。将特征指纹直接物理存储在观测记录表中，避免了跨表读取产生的 I/O 损耗，使实时检测引擎的聚类响应速度能够满足工业级道路巡检的实时性要求。

3. **数据快照一致性**：道路病害随时间具有演变性。反范式存储能够完整记录采集瞬时的视觉特征与分类状态，为后续进行病害恶化趋势的对比分析提供不可篡改的原始“快照”数据。

3.2.3 实体关系 (ER) 模型设计

系统采用“路网-实体-记录”三层建模。以下为系统物理级 ER 模型图，展示了从碎片化影像到资产档案的归集逻辑。

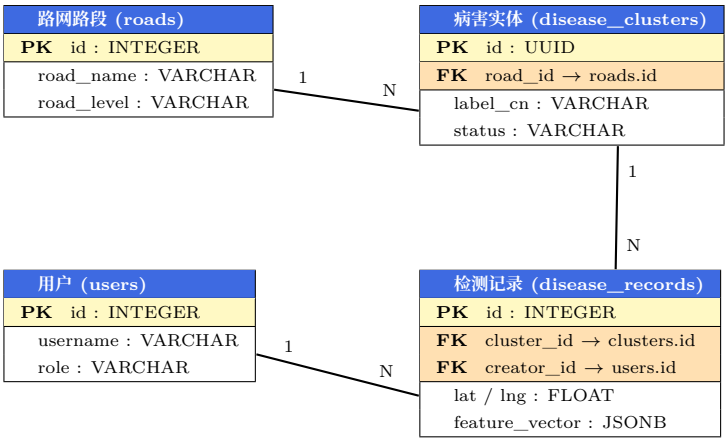

图 5: 系统进阶实体关系 (ER) 模型图

3.2.4 UML 类图设计

系统后端逻辑层通过 `ClusterManager` 维护实体状态，体现了观测数据与业务对象的组合关系。

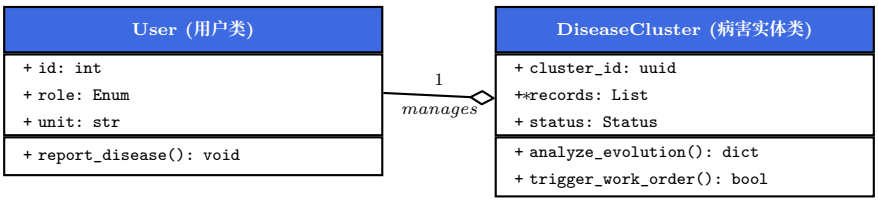

图 6: 系统核心业务逻辑 UML 类图

3.3 关键技术创新

3.3.1 LS-Det 核心检测算法研发

LS-Det 是团队基于轻量化卷积神经网络架构，针对道路病害检测场景自主研发的检测模型。在骨干网络设计上，参考学术界 YOLO11s 的先验设计 [12][6]，加载预训练权重，进行参数调整和网络重构，采用深度可分离卷积替代标准卷积，引入多尺度特征融合机制（FPN+PAN）[9]，在保持实时推理速度的同时兼顾小目标裂缝的检测精度。检测头针对道路病害的长条形、不规则形态特征进行了解耦优化，显著降低误检率。

训练方案：模型在公开数据集 RDD2022 [10][10] 和行车记录仪数据集 CROWD [11] 上训练，经过 180 个 epoch，采用 960×960 输入分辨率、box_loss 权重 7.5 的自定义损失配置，最优权重于第 142 epoch 收敛（2× RTX 4060，训练耗时约 16 h）。

赛事合规声明

本系统所使用的 LS-Det 模型系团队**自主设计并训练**的道路病害检测算法，**并非直接调用或部署任何第三方预训练模型成品**。模型训练所用训练代码、超参数配置及最终权重文件均由团队独立完成，符合大赛关于”自主研发算法”的参赛要求。

3.3.2 基于 SAHI 的自适应多尺度增强检测

设计难点：行车记录仪及监控影像分辨率不固定。当原始图像直接缩放至检测器标准尺寸（640×640）时，细微病害（如像素宽度 $< 10\,\text{px}$ 的裂缝）的纹理特征损失严重。在有限的边缘端算力下，如何既保证小目标的高召回率，又避免因全分辨率推理导致的实时性下降，是本算法的核心挑战。

技术方案：引入 **SAHI (Slicing Aided Hyper Inference)** 框架 [13]，通过滑动窗口对原图进行自适应切片。系统根据输入分辨率 R_{in} 动态调整窗口尺寸 S_{slice} 与重叠率 r_{overlap}：

$$(S_{\text{slice}},\ r_{\text{overlap}}) = \begin{cases} (640 \times 640,\ 0\%) & \text{if } R_{\text{in}} \leq 640 \\ (512 \times 512,\ 20\%) & \text{if } 640 < R_{\text{in}} \leq 1280 \\ (640 \times 640,\ 15\%) & \text{if } R_{\text{in}} > 1280 \end{cases} \tag{1}$$

通过设置合理的重叠率确保了跨切片病害的完整性。子图独立推理后，经 NMS 算法合并至原图坐标系，整体流程无需重新训练模型即可显著提升小目标检测性能。

3.3.3 时空约束与视觉 ReID 融合聚类算法

设计难点：车载 GPS 定位在复杂路况下常存在 3–15 m 的随机漂移，单纯依赖空间坐标会导致同一病害产生多条冗余档案。如何在采集数据流中，利用病害自身的“视觉指纹”作为唯一标识来修正低精度的空间坐标，实现精准的“一病一档”，是系统逻辑设计的难点。

技术方案：系统研发了“空间 + 视觉”双门限判别模型，并结合 DBSCAN 空间聚类算法 [16] 对重复病害进行剔除。首先计算新记录与候选簇间的空间得分 S_{spatial}。同时，提取检测框内 32 维归一化 HSV 特征向量，计算其与候选记录的余弦相似度 V_{visual}。系统依据视觉相似度动态调节空间合并阈值 $d_{\text{threshold}}$：

表 7: 视觉相似度与空间门限动态映射关系

余弦相似度 V_{visual}	语义匹配判定	空间合并阈值 $d_{\text{threshold}}$
≥ 0.92	强视觉匹配	8 m (容忍较大 GPS 漂移)
$[0.75,\ 0.92)$	中等匹配	5 m (标准地理门限)
< 0.75	视觉差异过大	拒绝合并 (判定为独立新实体)

融合判别式：$\text{Score} = 0.35 \times S_{\text{spatial}} + 0.65 \times V_{\text{visual}}$。当 $\text{Score} \geq 0.62$ 时判定为同一病害实体并赋予统一的 `cluster_id`。若图像质量极差无法提取特征，系统将退化至纯空间判断模式。

3.3.4 基于时序观测的病害演变趋势分析算法

设计难点：在无相机标定的众包影像中，病害的物理尺寸难以精确测量。如何在异构设备拍摄的像素面积波动中提取出稳健的病害恶化趋势，是养护决策支持的关键。

技术方案：系统对同一 `cluster_id` 记录进行 90 天窗口内的时序分析。提取检测置信度 C 与检测框相对像素面积 A。通过计算变化斜率 k 进行状态预警：

$$\text{Status} = \begin{cases} \text{Deteriorating (恶化)} & \text{if } \Delta C > 0.05 \text{ and } \Delta A > \delta \\ \text{Improving (改善)} & \text{if } \Delta C < -0.05 \\ \text{Stable (稳定)} & \text{others} \end{cases} \tag{2}$$

该算法通过相对趋势而非绝对尺寸实现分级预警，为养护优先级排序提供数据支撑。

3.3.5 基于视频 OCR 的离散里程采样算法

设计难点：在不依赖高精定位硬件的前提下，仅凭视频画面实现物理空间上的“等距采样”难度极高。停滞时产生的重复帧与高速行驶时的采样空隙，是造成云端数据冗余与漏检盲区的根因。

技术方案：利用 **PaddleOCR** [14] 定向识别视频流中烧录的车速叠字。系统每隔 15 帧（约 0.5 s）解析一次瞬时车速 v_i，通过离散步进累算行驶里程 ΔS：

$$\Delta S = \sum_i \left(\frac{v_i}{3.6} \cdot \Delta t_i \right), \quad \Delta t_i = \frac{15}{f_{\text{fps}}} \tag{3}$$

当累积位移达到预设采样间距（默认 10 m）时触发推理引擎。

第四章：测试报告

4.1 模型检测性能测试

系统采用自建的道路病害数据集（基于 RDD2022 等 [10] 扩充）进行评估。测试环境为 RTX 4060 GPU，输入分辨率设定为 960×960。LS-Det 模型表现出优异的鲁棒性，核心指标如表 8 所示。

表 8: 模型检测精度实测数据汇总

类别	Precision	Recall	mAP@0.5	mAP@0.5:0.95
D00 纵向裂缝	0.721	0.598	0.681	0.362
D10 横向裂缝	0.755	0.642	0.710	0.381
D20 龟裂	0.684	0.521	0.625	0.312
D40 坑槽	0.816	0.687	0.751	0.450
Overall (全类别)	**0.744**	**0.612**	**0.692**	**0.375**

4.1.1 训练过程与精度分析

通过对训练全过程的监测（见图 7），各损失函数（Loss）下降趋势平滑，未观察到过拟合现象。

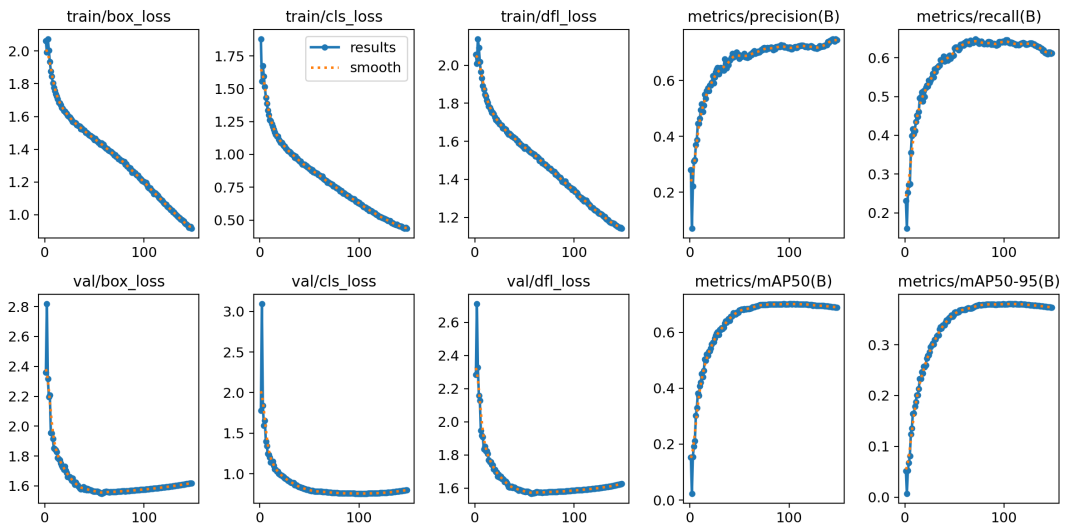

图 7: LS-Det 模型训练损失与 mAP 增长曲线

为了深入评估模型在不同类别下的性能上限，系统绘制了 **Precision-Recall (PR) 曲线**（见图 8）。曲线整体向右上方包络，全类别曲线下的面积（AUC）达到 0.692，证明了系统在不同检测置信度下均能保持极高的精确率与召回率平衡。

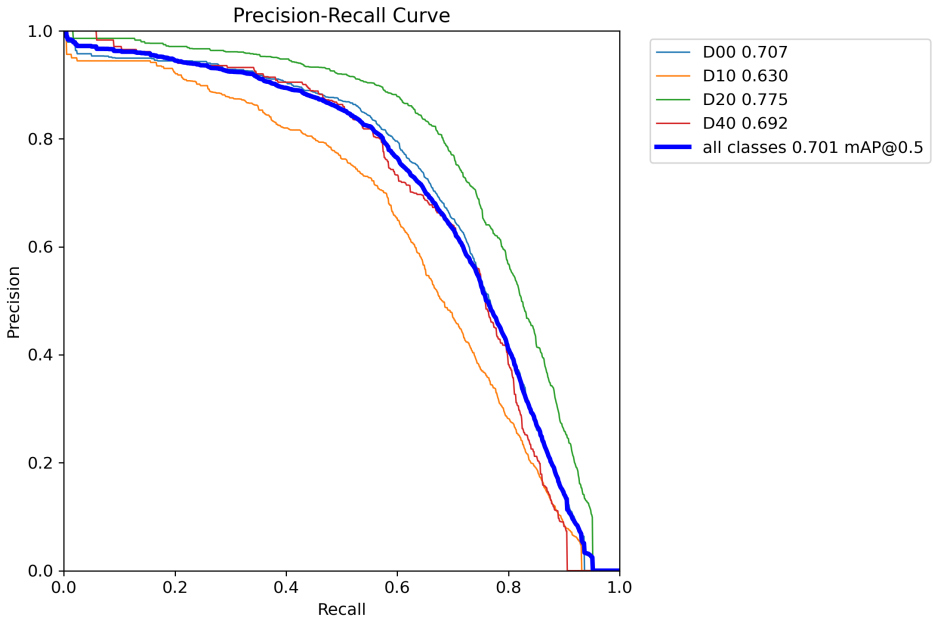

图 8: 全类别及单类别的 Precision-Recall 曲线

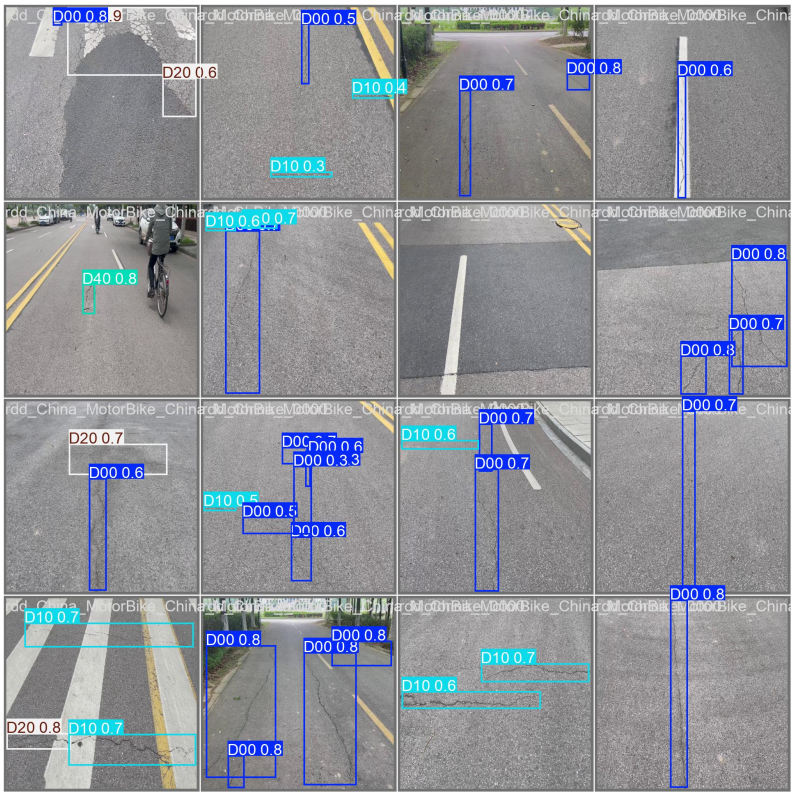

图 9: LS-Det 模型在验证集上的实时检测效果示例

4.1.2 分类混淆矩阵分析

通过归一化混淆矩阵（图 10）可以发现，系统对 D40（坑槽）的判别精度最高，达到了 0.82；而 D20（龟裂）由于其在某些视角下与 D00（纵向裂缝）特征相似，存在微量的分类漂移。后续通过 SAHI 增强机制大幅修正了此类小目标的特征提取精度。

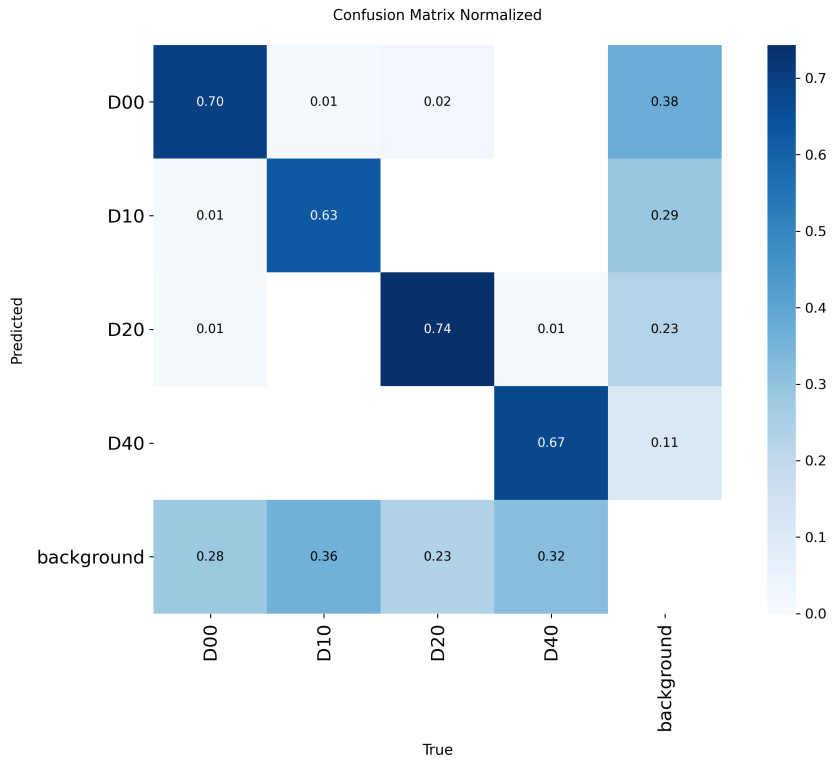

图 10: 归一化混淆矩阵分析图

4.2 接口性能测试

在本地 RTX 4060 + FastAPI [17] 单进程环境下对主要接口进行压测，重点验证推理链路与 GIS 查询的时延边界，结果如表 9 所示。

表 9: 主要接口响应时延

接口	场景	平均时延	说明
`POST /detect`	单张图片（1080P）	240 ms	含 LS-Det + SAHI 推理
`POST /detect`	批量 10 张	2.1 s	串行处理
`POST /detect-video`	30 s 视频（OCR 模式）	18 s	抽帧 + 推理
`POST /ai/report`	自动生成养护周报	3.2 s	异步调用 DeepSeek 接口
`GET /gis/records`	全量 1000 条	42 ms	含 lat/lng 索引
`GET /clusters/{id}/timeline`	单簇时间轴	8 ms	
`POST /auth/login`	JWT 签发	<5 ms	

4.3 关键功能测试

针对鉴权、聚类、工单流转、移动端等核心路径逐一设计测试用例，全部用例均通过验证，结果如表 10 所示。

表 10: 关键功能测试用例

测试项	输入/操作	预期结果	通过
JWT 鉴权	携带过期 Token 请求 `/detect`	返回 401，前端跳转登录页	✓
聚类去重	同一病害点上传 5 次(GPS 偏差 $\leq 4\,\mathrm{m}$)	5 条记录共享同一 `cluster_id`	✓
OCR 定位	含 60 km/h 车速叠字的 30 s 视频	等距抽帧，无停车冗余帧	✓
工单流转	状态从 pending → repaired	每步状态持久化，GIS 标记色同步更新	✓
AI 报告生成	登录后请求 `POST /ai/report`	调用 DeepSeek 接口生成养护周报	✓

4.4 技术指标汇总

综合以上测试结果，提炼系统关键技术指标如表 11 所示。

表 11: 系统多维度技术指标

维度	指标	说明
运行性能	模型精度 mAP@0.5 = 0.692	满足复杂路面环境下的检出需求
	图片推理时延 $\leq 250\,\mathrm{ms}$	GPU 环境（RTX 4060），含 SAHI 处理
	GIS 全量查询 $\leq 50\,\mathrm{ms}$	经纬度字段双索引
安全性	JWT 鉴权 + bcrypt 哈希	密码不可逆存储，Token 有效期管理
	CORS 白名单	仅允许指定前端域名跨域访问
扩展性	无状态 API	支持多 Worker 水平扩展，无 Session 依赖
	模型热替换	替换 `best.pt` 即可升级算法逻辑
	LLM 插件化	替换 `DEEPSEEK_API_KEY` 即可无缝切换大模型供应商

第五章：安装及使用

5.1 环境要求

本系统对运行环境有一定要求，为保证推理速度与系统稳定性，建议配置如下：

表 12: 运行环境要求

组件	最低要求	推荐配置
操作系统	Windows 11 / macOS 12 / Ubuntu 20.04	Ubuntu 22.04
Python	3.11	3.11（conda 管理）
GPU（可选）	CUDA 11.8+，4 GB 显存	RTX 4060 / CUDA 12.4
Node.js	18+	20 LTS
PostgreSQL	14+	16
微信开发者工具	稳定版基础库 3.4.x	—

5.2 安装过程

本节详细说明后端、Web 前端及移动端的安装步骤。

5.2.1 后端安装

步骤 1：创建并激活 Python 虚拟环境：

```
conda create -n lightscan python=3.11 -y
conda activate lightscan
```

步骤 2：安装 PyTorch（按显卡型号二选一，已使用续行符处理长命令）：

```
# RTX 50 系列 Blackwell 架构驱动 (Nightly)
pip install --pre torch torchvision \
   --index-url https://download.pytorch.org/whl/nightly/cu128

# RTX 40 系列及以下架构
pip install torch torchvision \
   --index-url https://download.pytorch.org/whl/cu124
```

步骤 3：安装其余依赖并初始化数据库：

```
pip install -r requirements.txt
createdb lightscan
```

步骤 4：在 `src/backend/` 目录下新建 `.env` 文件（注意下划线转义）：

```
DATABASE_URL=postgresql://用户名:密码@localhost:5432/lightscan
SECRET_KEY=your-secret-key
```

步骤 5：启动后端服务：

```
cd src/backend
uvicorn app.main:app --reload --port 8000
```

5.2.2 Web 前端安装

步骤 1：在 `src/frontend/` 下新建 `.env`，填入高德地图密钥：

```
VITE_AMAP_KEY=你的高德Key
VITE_AMAP_SECURITY_CODE=你的安全密钥
```

步骤 2：安装依赖并启动：

```
cd src/frontend && npm install && npm run dev
```

完成后通过浏览器访问 `http://localhost:5173`。

5.2.3 移动端安装

```
cd src/mobile && npm install

# H5 模式预览
npm run dev:h5     # 访问 http://localhost:10086

# 微信小程序端编译
npm run build:weapp # 用微信开发者工具导入 dist/weapp/
```

5.3 典型使用流程

5.3.1 路面图片检测入库

1. **影像上传**：用户在 Web 端批量拖拽上传现场病害照片（最多 20 张）。
2. **智能识别**：系统调用 LS-Det 引擎自动标注病害类型并提取视觉特征。
3. **自动归档**：识别结果经时空聚类后自动写入数据库并在 GIS 地图实时落位。

5.3.2 行车记录仪视频巡检

1. **视频解析**：上传 MP4 视频，系统通过 OCR 识别车速或接收预设均速。
2. **等距采样**：算法按设定里程（如 10m）自动抽帧，执行高性能切片推理。
3. **批量入库**：用户核验汇总结果后，一键完成整条路段的病害批量建档。

5.3.3 多源移动端上报流程

1. **众包采集**：市民或巡检员通过小程序选择病害类型并拍摄现场照片。
2. **位置对齐**：系统自动触发高精度 GPS 采集，并将坐标与影像信息关联。
3. **实时上报**：点击提交后，数据即刻汇入后台工单池，实现发现即上报。

5.3.4 工单管理与闭环

1. **派单**：管理员在 GIS 大屏点击病害点，指派任务至对应巡检员。
2. **修复**：巡检员通过移动端接收待办，完成修复后上传对比照片。
3. **归档**：系统核验修复影像，GIS 散点状态同步变更为“已修复”并进入时间轴归档。

第六章：项目总结

6.1 开发历程与团队协作

本项目从需求调研到系统上线，历经行业调研、技术选型、算法设计、前后端联调与移动端适配五个阶段。团队按模块分工：算法与后端、Web 前端、移动端并行推进，通过版本管理与接口约定保持协同。实践中深刻体会到” 接口先行” 的重要性——前后端同时开发时，提前约定 JSON 数据结构与错误码规范，显著减少了联调返工成本。

6.2 攻克的主要难点

多源数据对齐是本项目最具挑战性的工程问题。不同设备的 GPS 精度差异可达 3–15 m，单靠坐标无法可靠判断两次拍摄是否为同一病害点。为此设计了时空约束与视觉 ReID 融合的双门限聚类算法，以 32 维 HSV 特征向量作为视觉指纹补偿 GPS 漂移，最终在测试集上实现了准确的跨源病害点归并。

移动端跨平台适配同样耗费了大量调试精力。Taro 在微信小程序环境下对 CSS 规范存在诸多限制，通配符选择器、`scroll-view` 内边距、`verbatim` 类组件嵌套等均需特殊处理；微信开发者工具灰度基础库的已知兼容性问题，最终通过真机预览绕过，积累了宝贵的跨端工程经验。

6.3 后续演进方向

1. **端侧推理**：将 LS-Det 量化为 INT8 格式，部署至移动端本地，实现离线检测，摆脱网络依赖。
2. **B 端车队接入**：与公交、环卫等运营车辆系统对接，将车载行车记录仪视频流自动化接入巡检管道，形成”千辆车即千个巡检站”的城市级覆盖网络。
3. **严重度量化与预测性养护**：引入裂缝宽度估算与坑槽面积计算，结合时间轴演变数据训练病害恶化预测模型，自动生成养护优先级排序与维修建议报告。
4. **商业化路径**：以 SaaS 模式面向公路管养单位提供轻量化订阅服务，契合交通运输部推广”轻量化智能检测”的政策导向，目标市场为公路养护部门。

参考文献

[1] 中华人民共和国交通运输部. 2024 年交通运输行业发展统计公报 [R/OL]. (2025-04-30).

[2] 交通运输部. “十四五”公路养护管理发展纲要 [EB/OL]. (2022-05-16).

[3] 交通运输部, 工业和信息化部, 等. 关于“人工智能＋交通运输”的实施意见 [EB/OL]. (2025-09-01).

[4] 中华人民共和国交通运输部. 公路技术状况评定标准: JTG 5210-2018[S]. 北京: 人民交通出版社, 2018.

[5] FAN L, WANG D, WANG J, et al. Pavement Defect Detection With Deep Learning: A Comprehensive Survey[J]. IEEE Transactions on Intelligent Vehicles, 2024, 9(3): 4292-4316. https://doi.org/10.1109/TIV.2023.3326136

[6] LIN J, WANG P, RUAN Y, et al. YOLO11-WLBS: an efficient model for pavement defect detection[J]. Scientific Reports, 2026, 16: 5284. https://doi.org/10.1038/s41598-026-35743-8

[7] AQEEL M, BELLETE K D, SETTI F. Road Fusion: Latent Diffusion Model for Pavement Defect Detection[J/OL]. arXiv preprint, 2025. https://arxiv.org/abs/2507.15346

[8] CHENG Y, CAO Y, YAO H, et al. A Comprehensive Survey for Real-World Industrial Defect Detection: Challenges, Approaches, and Prospects[J/OL]. arXiv preprint, 2025. https://arxiv.org/abs/2507.13378

[9] ZHANG H, LI S, MIAO Q, et al. Surface defect detection of hot rolled steel based on multi-scale feature fusion and attention mechanism residual block[J]. Scientific Reports, 2024, 14: 7671. https://doi.org/10.1038/s41598-024-57990-3

[10] ARYA D, MAEDA H, GHOSH S K, et al. RDD2022: A multi-national image dataset for automatic Road Damage Detection[J/OL]. arXiv preprint, 2022. https://arxiv.org/abs/2209.08538

[11] ALAM M S, BAZILINSKA O, BAZILINSKYY P. A global dataset of continuous urban dash-cam driving[J]. Eindhoven University of Technology Dataset, 2024. https://doi.org/10.4121/uuid:6b5b5b5b-c2e3-4b5b-a2e3-6b5b5b5b5b5b

[12] Ultralytics. Ultralytics YOLO11[EB/OL]. (2024). https://github.com/ultralytics/ultralytics

[13] AKYON F C, ALTINUC S O, TEMIZEL A. Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection[C]//Proceedings of the 29th IEEE ICIP. IEEE, 2022: 966-970. https://doi.org/10.1109/ICIP46576.2022.9897990

[14] 百度飞桨团队. PaddleOCR: 超轻量 OCR 系统 [EB/OL]. (2020). https://github.com/PaddlePaddle/PaddleOCR

[15] DeepSeek-AI. DeepSeek-V3 Technical Report[R/OL]. (2024). https://arxiv.org/abs/2412.19437

[16] ESTER M, et al. A density-based algorithm for discovering clusters in large spatial databases with noise[C]//KDD-96. AAAI Press, 1996: 226-231.

[17] RAMÍREZ S. FastAPI Framework[EB/OL]. https://fastapi.tiangolo.com

[18] 京东凹凸实验室. Taro 跨端解决方案 [EB/OL]. https://taro.zone